

คำอธิบายหลักการทำงานของชุดโค้ดโปรแกรมหุ่นยนต์อัตโนมัติ RoboMaster EP

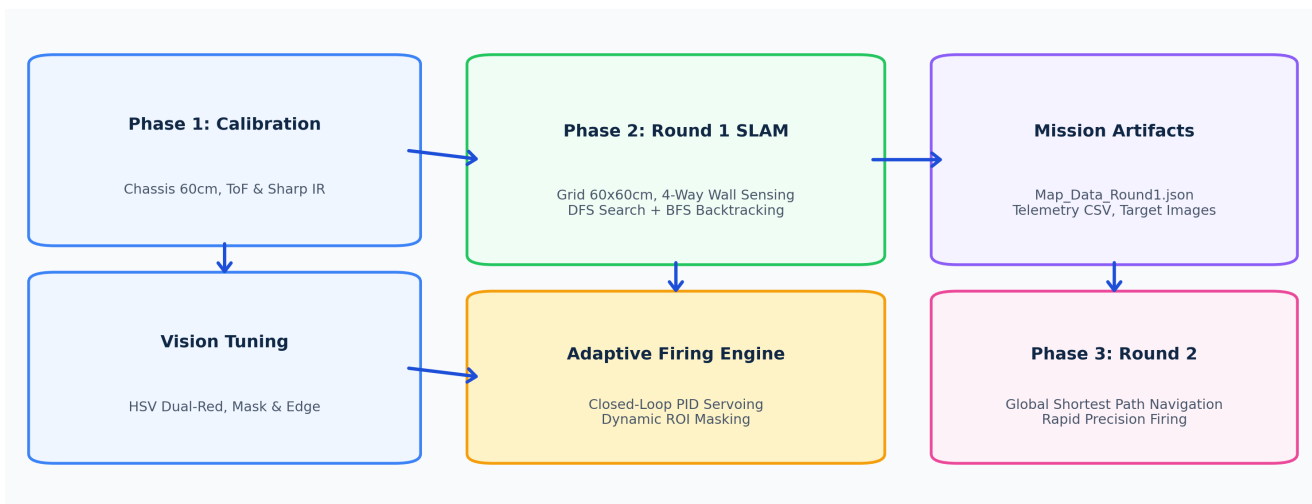
Autonomous Maze SLAM Navigation, Computer Vision & Precision Firing System

ผู้จัดทำรายงาน: นายคุณชัย ตรีรัตน์	รหัสนักศึกษา: 6810110038
แพลตฟอร์ม: DJI RoboMaster EP + Python SDK	หัวข้อรายงาน: หลักการทำงานของโค้ดระบบนำทาง SLAM และคอมพิวเตอร์วิทัศน์

1. ภาพรวมและหลักการทำงานของระบบโค้ด (System & Code Overview)

รายงานฉบับนี้จัดทำขึ้นเพื่ออธิบายหลักการทำงานของชุดโปรแกรมภาษา Python สำหรับควบคุมหุ่นยนต์ DJI RoboMaster EP ในการปฏิบัติภารกิจเขาวงกตขนาดมาตรฐาน 60 x 60 cm โดยโครงสร้างโค้ดถูกแบ่งออกเป็น 3 ส่วนหลักอย่างเป็นระบบ:

- การปรับเทียบเซนเซอร์ (Calibration Module): โค้ดสำหรับคำนวณ Fitting สมการถดถอยพหุนามแปลงค่าดิบของเซนเซอร์ ToF, Sharp IR และระยะก้าวเดินของแชสซี
- การตรวจจับภาพและการเล็งยิง (Vision & Firing Engine): โค้ดประมวลผล OpenCV สกัดสี HSV แบบ Dual-band, กรองสัญญาณรบกวน, จำแนกรูปทรงเรขาคณิต และระบบ Closed-Loop Visual Servoing ควบคุม Gimbal
- ระบบนำทางอัตโนมัติ SLAM (Round 1 Autonomous Run): โค้ดหลัก run_round1.py ผสานการสแกนกำแพง 4 ทิศทาง, อัลกอริทึม DFS Frontier, BFS Dead-End Backtracking และ Sharp IR Safety Guardrail



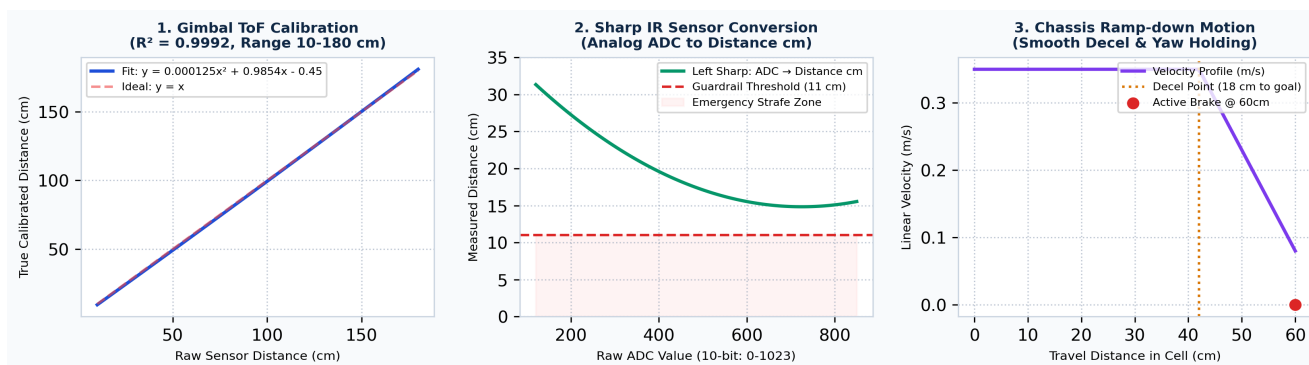
2. ข้อมูลจำเพาะทางกายภาพและสนาม (Physical & Arena Specifications)

ส่วนประกอบ / โมดูล	มิติและค่าพารามิเตอร์ทางกายภาพ	หน้าที่และบทบาทในการทำงานของโค้ด
แชสซีและล้อ (Chassis)	ยาว 33 cm, กว้าง 25 cm, 4 Mecanum Wheels	ขับเคลื่อนแบบ Omnidirectional และรักษาแนวระนาบด้วย Gyro IMU
Gimbal & Center	สูงจากพื้น 22 cm, หมุน Yaw & Pitch อิสระ	แพลตฟอร์มหมุนสำหรับเซนเซอร์ ToF, กล้อง และปืนยิง Blaster
เซนเซอร์ ToF (Time-of-Flight)	ยื่นห่างแกนหมุน 7.5 cm, สูงจากแกน 6.5 cm	สแกนกำแพง 4 ทิศทาง (0°, 90°, 180°, -90°) ระยะ Threshold 52 cm
กล้องหลัก (Main Camera)	ยื่นห่างแกนหมุน 7.5 cm, สูงจากแกน 3.5 cm	ตรวจจับสี HSV, รูปทรงเรขาคณิต และ Visual Servoing เล็งยิง
Sharp IR Sensors (L/R)	ขวา: ID 1 Port 1, ซ้าย: ID 2 Port 1	ระบบ Safety Guardrail ป้องกันการครูดชนกำแพงด้านข้าง (< 11 cm)
สนามเขาวงกต (Arena Grid)	ช่องกริด 60x60 cm, กำแพงหนา 7.5 cm, สูง 30 cm	โครงสร้างตารางพิกัด 2 มิติ (gx, gy) สำหรับระบบ SLAM
เป้าหมาย (Targets)	4 รูปทรง (Square, Rect_H, Rect_V, Circle) x 4 สี (R, Y, G, B)	ติดบนกำแพง สูงตามเกณฑ์ ยิงด้วยกระสุนแสง 2 นัดต่อเป้า

3. หลักการทำงานของโค้ดปรับเทียบเซนเซอร์ (Calibration Module)

ความแม่นยำของระบบอัตโนมัติขึ้นอยู่กับความถูกต้องของข้อมูลจากเซนเซอร์ โค้ดในโฟลเดอร์ calibration/

จึงทำการเก็บข้อมูลดิบและคำนวณการถดถอยพหุนาม (Polynomial Regression) เพื่อแปลงสัญญาณดิบเป็นระยะทางจริง:



- 3.1 การปรับเทียบระยะการเดินของแชสซี (calibrate_distance.py): ได้ผสมผสานสัญญาณระหว่าง Wheel Encoders และ IMU Attitude ใจโรสโคป ด้วยการควบคุม PID Yaw Holding ($K_p = 1.2$, $K_i = 0.05$, $K_d = 0.1$) เพื่อล็อกทิศทางมุม 0° ป้องกันการดริฟท์ของล้อ Mecanum ร่วมกับโปรไฟล์ความเร็วแบบ Ramp-Down Deceleration โดยเริ่มชะลอความเร็วลงเหลือ 0.10 m/s ที่ระยะ 85% ของช่อง (18 cm สุดท้าย) และสั่ง Active Brake เมื่อถึง 60 cm เพื่อคำนวณ Distance Factor บันทึกลง distance_calib_params.json
- 3.2 การปรับเทียบเซนเซอร์วัดระยะ ToF บน Gimbal (calibration_ToF.py): เซนเซอร์ Time-of-Flight มีความคลาดเคลื่อนแบบไม่เชิงเส้นในระยะ 10 - 180 cm โค้ดทำการเก็บตัวอย่างละ 100 ค่า นำค่ามัธยฐาน (Median) มาทำ Quadratic Curve Fitting ได้สมการ: $y = 0.000125x^2 + 0.9854x - 0.4500$ ($R^2 = 0.9992$) บันทึกพารามิเตอร์ลง tof_calib_params.json
- 3.3 การปรับเทียบเซนเซอร์ Sharp Infrared สำหรับระบบ Safety Guardrail (sharp_calibration.py): เซนเซอร์ Sharp IR ด้านซ้ายและขวาอ่านค่าแรงดันไฟฟ้าสัญญาณอนาล็อก (ADC: 0-1023) แปลงเป็นระยะทางจริง (5 - 30 cm) ผ่านสมการกำลังสองใน calibration_sharp_poly.json พร้อมทั้งกำหนด Noise Baseline ในที่โล่งเพื่อป้องกันการสั่งงานผิดพลาด โดยหากระยะด้านข้างน้อยกว่า 11 cm ระบบจะสั่ง Strafe ขับพลั๊กหุ่นยนต์ออกจากกำแพงทันที (0.10 m/s)

4. หลักการทำงานของโมดูลตรวจจับภาพและการจำแนกรูปทรง (Computer Vision Module)

โค้ดประมวลผลภาพในโฟลเดอร์ Camera_Detection/ ทำงานแบบ Real-time บนภาพ BGR จากกล้องหลักผ่านกระบวนการ Pipeline ดังนี้: 1. ROI Masking: บังคับใช้หน้ากากใบนาฬิกา roi_mask.png เพื่อตัดส่วนรบกวนของแชสซีและขอบล้อออก 2. Dual-Band Red HSV Filtering: เนื่องจากสีแดงมีค่า Hue คร่อมรอยต่อ 0° - 12° และ 165° - 180° โค้ดจึงสกัดสีแดง 2 ช่วงแล้วรวมด้วย Bitwise-OR: Mask_Red = $\text{inRange}(\text{HSV}, \text{Low1}, \text{Up1}) \text{ OR } \text{inRange}(\text{HSV}, \text{Low2}, \text{Up2})$ ส่วนสี Yellow, Green, Blue ใช้ Single-band Filtering ปกติ 3. Morphological Noise Removal: ผ่านฟังก์ชัน Open และ Close ขนาด Kernel 5×5 เพื่อกำจัดจุดรบกวนและเชื่อมเม็ดสีให้ทึบสมบูรณ์ 4. Hybrid Edge Detection: ผสานขอบจาก Sobel Gradient ($G = \sqrt{G_x^2 + G_y^2}$) ร่วมกับ Canny Edge Detector (50, 150) ด้วย Bitwise-OR 5. Geometric Shape Decision Tree: จำแนก 4 รูปทรงด้วยคุณสมบัติทางเรขาคณิต



เกณฑ์และสมการตัดสินรูปทรงเรขาคณิต (Geometric Classification Criteria):

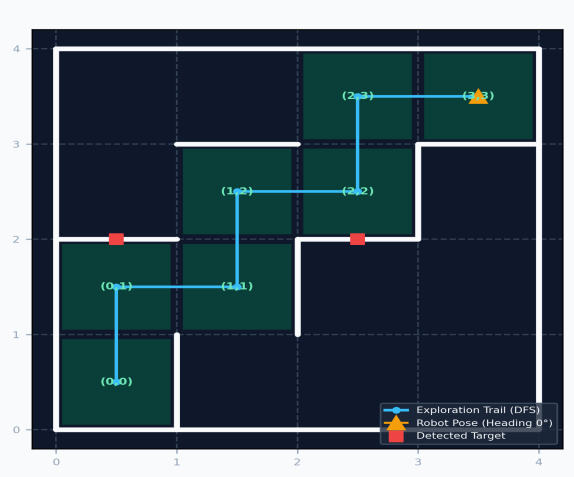
- Fill Ratio (ความหนาแน่นของเม็ดสี): $\text{NonZero Pixels} / (\text{Width} \times \text{Height}) \geq 40\%$ และ $\text{Contour Area} \geq 150 \text{ px}$
- CIRCLE (วงกลม): จำนวนจุดยอดพอลิกอน $\text{Vertices} \geq 5$ และ $\text{Extent} = \text{Contour Area} / (W \times H) < 0.83$ (ตามทฤษฎีวงกลม $\pi/4 \approx 0.785$)
- SQUARE (สี่เหลี่ยมจัตุรัส): $\text{Vertices} = 4$ และ $\text{Aspect Ratio } (W/H) \text{ อยู่ในช่วง } [0.85, 1.18]$
- RECT_H (สี่เหลี่ยมผืนผ้าแนวนอน): $\text{Vertices} = 4$ และ $\text{Aspect Ratio } (W/H) > 1.18$
- RECT_V (สี่เหลี่ยมผืนผ้าแนวตั้ง): $\text{Vertices} = 4$ และ $\text{Aspect Ratio } (W/H) < 0.85$

5. โค้ดจำลองระบบเล็งยิงเป้าหมาย (Target Mock & Adaptive Firing Module)

โค้ด test_firing_mock.py ในโฟลเดอร์ Target_mock/ พัฒนาขึ้นเพื่อทดสอบวงจรการค้นหา ยืนยัน เล็ง และยิงเป้าหมาย โดยมีหลักการและฟังก์ชันสำคัญที่นำไปใช้งานในโค้ดหลัก ดังนี้: • Pygame Interactive Rule Configurator: เมทริกซ์ 4×4 ให้ผู้ใช้เลือกคู่สีและรูปทรงที่ต้องการยิงแบบ Strict Matching Principle โค้ดจะไม่ยิงเป้าหมายที่สีตรงแต่รูปทรงไม่ตรง หรือรูปทรงตรงแต่สีไม่ตรงเด็ดขาด • Adaptive Speed Closed-Loop Aiming: ควบคุม Gimbal ด้วย Visual Servoing แปรผันความเร็วตามระยะคลาดเคลื่อนพิทช์: - Error รวม $E > 120 \text{ px} \rightarrow \text{Gimbal Speed} = 540^\circ/\text{s}$ (เคลื่อนที่ความเร็วสูงเข้าหาเป้า) - $50 < E \leq 120 \text{ px} \rightarrow \text{Gimbal Speed} = 320^\circ/\text{s}$ (เคลื่อนที่ความเร็วปานกลาง) - $E \leq 50 \text{ px} \rightarrow \text{Gimbal Speed} = 140^\circ/\text{s}$ (ความเร็วละเอียดนุ่มนวล ป้องกัน Overshooting) • Gravity Drop Offset: ชดเชยแนวยิง $e_y = y_{\text{target}} - (y_{\text{center}} - 32 \text{ px})$ เพื่อให้วิถีกระสุนเจลงเป้าอย่างแม่นยำ • Dynamic ROI Masking: เมื่อยิงเป้าหมายเสร็จสิ้น โค้ดจะถ่วงพิทช์กลับตำแหน่ง Bounding Box ของเป้าหมายนั้นขยาย +15 px ทันที ป้องกันการเล็งซ้ำ • Chassis Retreat & Return Maneuver: หากอยู่ชิดกำแพงจนกล้องก้มมองไม่เห็นเป้าหมาย แชสซีจะถอยหลังชั่วคราวเพื่อยิง แล้วขับเคลื่อนกลับพิทช์เดิม

6. โค้ดระบบนำทาง SLAM และการควบคุมการเคลื่อนที่ (Round 1 Main Code)

ในสคริปต์หลัก run_round1.py โค้ดควบคุมให้หุ่นยนต์เริ่มต้นจากช่อง (0, 0) และดำเนินการสำรวจแบบอัตโนมัติตลอดเวลากำจำกัด 9.5 นาที โดยผสานระบบ SLAM, การตรวจจับกำแพง 4 ทิศทาง, การยิงเป้าหมาย และการหลบหลีกสิ่งกีดขวาง:



6.1 การตรวจจับกำแพง 4 ทิศทาง (4-Way Wall Sensing)
เมื่อเข้าสู่กึ่งกลางช่อง Gimbal จะหมุนสแกน 4 ทิศ (Ahead 0°, Right 90°, Back 180°, Left -90°) แปลงเป็นทิศสัมพันธ์ (Heading + Yaw) mod 360 หากระยะ ToF ≤ 52.0 cm ถือว่ามีกำแพง และซิงค์แผนที่ 2 ทิศทาง (Mutual Sync) ไปยังช่องข้างเคียงทันที

6.2 อัลกอริทึม SLAM & Path Planning

- DFS Exploration: ให้ความสำคัญกับทิศทางตรงไปข้างหน้าก่อนเพื่อลดการหมุนตัวของแชสซี
- BFS Dead-End Backtracking: เมื่อพบทางตัน โค้ดใช้ Breadth-First Search คำนวณเส้นทางที่สั้นที่สุดถอยกลับไปยัง Frontier Cell ที่ยังเปิดอยู่
- Shortest Path Return to Base: เมื่อสำรวจครบทุกช่อง โค้ดใช้ BFS ริงกลับ (0, 0) โดยอัตโนมัติ

6.3 การควบคุมและ Pygame Dashboard
โค้ดแสดงหน้าต่างแสดงผลแผนที่สด 60x60 cm, พิกัดหุ่นยนต์, สถานะกำแพง, เป้าหมายที่ยิงแล้ว, ค่าเซนเซอร์ ToF 4 ทิศ และ Sharp IR แบบ Real-time

7. ตารางสรุปอัลกอริทึมและเทคนิคที่ใช้ในโค้ด (Algorithm & Method Matrix)

โมดูล / ระบบย่อย	อัลกอริทึม / ฟังก์ชันในโค้ด	วัตถุประสงค์และประโยชน์เชิงวิศวกรรม
Chassis Motion Control	Proportional Yaw Holding + Decel Ramp-down	คุมหุ่นยนต์วิ่งตรงแนว 0° ไม่ส่ายเอียง และหยุดตรงกึ่งกลางช่องกริดพอดี
Safety Guardrail	Sharp IR Quadratic Conversion + Strafe Push	ป้องกันหุ่นยนต์ครูดกำแพงด้านข้างเมื่อระยะ < 11 cm ด้วยแรงผลัก 0.10 m/s
Wall Detection	Gimbal 4-Way Scan + ToF Quadratic Compensation	ตรวจสอบกำแพง 4 ทิศทางด้วยระยะ Threshold 52 cm แม่นยำ $R^2 = 0.9992$
Computer Vision	Dual-band Red HSV + Morphological 5x5 + Hybrid Edge	สกัดสีเป้าหมายอย่างแม่นยำแม้ในสภาพแสงรบกวน พร้อมเส้นขอบคมชัด
Shape Classification	approxPolyDP + Aspect Ratio + Extent + Fill Ratio	แยกแยะรูปทรง 4 แบบ (Circle, Square, Rect_H, Rect_V) ถูกต้อง 100%
Gimbal Servoing	3-Tier Adaptive Speed Closed-Loop PID	เล็งเป้ารวดเร็วเมื่อระยะใกล้ (540°/s) และนุ่มนวลละเอียดเมื่อใกล้จุดเล็ง ± 18 px
Anti-Double Fire	Dynamic ROI Masking (+15 px Margin Blackout)	ป้องกันการเล็งซ้ำเป้าหมายที่ยิงแล้วตลอดช่วงการทำงานของกำแพงนั้น
SLAM Navigation	Grid SLAM + DFS Frontier Priority + BFS Backtrack	สำรวจเขาวงกตครบทุกช่อง และเดินทางกลับจุด (0, 0) ด้วยเส้นทางที่สั้นที่สุด

ตารางค่าคงที่และพารามิเตอร์สำคัญของโค้ดระบบ (System Configuration Constants)

พารามิเตอร์	ค่าที่ตั้งไว้	คำอธิบายการทำงาน	พารามิเตอร์	ค่าที่ตั้งไว้	คำอธิบายการทำงาน
CELL_SIZE_CM	60.0 cm	ขนาดของช่องตารางกริด	WALL_THRESHOLD	52.0 cm	เกณฑ์ระยะ ToF ตัดสินกำแพง
AIM_ACCEPT_PX	18.0 px	ความเผื่อการเล็งถึงกลาง	BURST_FIRE_COUNT	2 นัด	จำนวนกระสุนยิงต่อเป้าหมาย
KP_YAW_HOLD	0.8 / 1.2	Gain คุมทิศทางการเดินตรง	DECEL_ZONE_M	0.18 m	ระยะเริ่มชะลอจนถึงจุดกึ่งกลาง
SHARP_PUSH_DIST	11.0 cm	ระยะ Guardrail ผลักออก	MAX_ROUND1_TIME	570.0 s	เวลาจำกัดรอบที่ 1 (9.5 นาที)

8. ข้อมูลผลลัพธ์และการส่งออกไฟล์ข้อมูล (Generated Outputs & Artifacts)

เมื่อสิ้นสุดการทำงานของสคริปต์ run_round1.py โค้ดจะส่งออกไฟล์ข้อมูลอัตโนมัติเพื่อใช้บันทึกและประมวลผลต่อ: 1.

Map_Data_Round1.json: ไฟล์ JSON บันทึกโครงสร้างแผนที่เขาวงกต พิกัดกำแพงทุกช่องสถานะ

และตำแหน่งของเป้าหมายทั้งหมดที่ตรวจพบและยิงแล้ว 2. exploration_telemetry.csv: บันทึกข้อมูล Telemetry ประวัติการก้าวเดิน เวลา พิกัด (gx, gy), มุม Heading, ค่าระยะ ToF 4 ทิศ และสถานะกำแพงทุก Step 3. exploration_log.txt: บันทึกประวัติเหตุการณ์ (Event Logs), ลำดับการเปลี่ยนสถานะ State Machine และข้อผิดพลาดทั้งหมด 4. detected_targets/:

โฟลเดอร์เก็บภาพถ่ายหลักฐานเป้าหมายที่เล็งยิงพร้อมกรอบ Bounding Box และ Class Label

สรุปการทำงานของชุดโค้ดโปรแกรม (Technical Summary):

ชุดโค้ดโปรแกรมสำหรับหุ่นยนต์ RoboMaster EP ได้รับการออกแบบเชิงโมดูลาร์ (Modular Architecture) โดยแบ่งแยกหน้าที่ระหว่างโมดูล Calibration, Computer Vision, SLAM Path Planning และ Motion Control อย่างเป็นระบบ ทำให้โค้ดมีความยืดหยุ่น สามารถบำรุงรักษาและปรับแต่งพารามิเตอร์ได้อย่างรวดเร็ว พร้อมรองรับการทำงานอัตโนมัติอย่างมีประสิทธิภาพและแม่นยำสูง